A3-PS002-TC140

ConsentGuard: Purpose-Based Consent Recording & Withdrawal Prototype

Team Name : No means No
Team Members:
Md Samir Khan - md2102172@st.jmi.ac.in
Aliza Parvez Yousuf- alizaparvez45@gmail.com

AIORI Hackathon 3

Problem Description

Abstract ConsentGuard is a purpose-based consent recording and withdrawal prototype for a synthetic fitness service (FitNest), built on MERN stack. It records purpose, notice version, grant and withdrawal for three data-sharing purposes. A mock downstream API(Partnerhub)checks the live consent state on every request using short-lived signed tokens and a consent -version counter,so withdrawn or outdated consent is denied on the next request. A hash-chained audit trail and a 20-test suite provides verifiable technical evidence.

1 Blanket consent

Traditional: One "Agree" button allows all data usage, and users don’t know the purpose.
ConsentGuard: Separate consent per purpose; the data consumer must name the purpose for every request.

2 Opaque notice versions

Traditional: Policies change and users are not informed.
ConsentGuard: Track notice versions and deny access until the user re-consents.

3 Delayed withdrawal

Traditional: Data keeps flowing after consent is removed, which can lead to unauthorized use.
ConsentGuard: Access stops instantly, in real time.

4 No verifiable evidence

Traditional: No proper records of consent actions, so it is hard to prove rules are followed.
ConsentGuard: Tamper-evident , hash-chained audit trail. Any edited or deleted record breaks the chain and is detected.

India’s DPDP Act 2023 requires specific, informed consent and withdrawal as easy as granting it [1].

Background

Purpose-limited consent

Consent must be tied to a specific purpose. The DPDP Act 2023 requires consent to be free, specific, informed and unambiguous (s.6) [1], and GDPR requires purpose limitation and withdrawal as easy as giving consent (Art. 5(1)(b), 7(3)) [3]. ConsentGuard records a separate consent for each of three purposes.

Notice version (nv)

Consent is only meaningful for the notice the user actually saw [1][5]. Each notice has a version number (nv). Consent is stored with the nv it was given against. If a material change is published, consent for the affected purpose is treated as outdated and the user is asked again

Consent version (cv)

A counter kept per user and purpose. It increases by one on every grant or withdrawal (grant → cv=1, withdraw → cv=2). A token carries the cv it was issued with. If it no longer matches the live cv, the request is denied.

Why tokens alone fail

A signed JWT stays valid until it expires [6][7], so withdrawal alone cannot stop a token already issued. Checking with the server on every request adds an extra network call [8][9]. ConsentGuard uses short-lived tokens verified locally, plus a fast consent-counter (cv) check against a local cache.

Evidence must be tamper-evident

Under GDPR, Controllers must be able to demonstrate consent [3], and the DPDP Rules require logs to be retained for at least one year [2]. Each audit event stores the hash of the previous one, so editing or deleting a record breaks the chain and the verifier detects it [10][11]. This detects tampering. It does not prevent it.

Key terms

Purpose: (P1 analytics, P2 personalisation, P3 research sharing)
nv / cv: notice version / consent counter
Assertion token: a 5-minute signed token proving one user's consent for one purpose User = data principal, FitNest = data fiduciary, PartnerHub = mock downstream consumer.

Scope note: technical consent evidence, not legal certification.

Literature & RFC Review

Source	Key idea	Gap / how ConsentGuard differs
DPDP Act 2023 (§5–6) and Rules 2025 [1][2]	Notice, purpose-limited consent, withdrawal as easy as granting, processors stop within a reasonable time	The law says what, not how. We implement enforceable state at the consumer.
GDPR Art. 5(1)(b), 7 [3]	Purpose limitation, withdrawal as easy as consent,Controller must demonstrate consent(Art.7(1)).	Same gap, no technical enforcement design.
Kantara Consent Receipt v1.1 [4]	Machine-readable record of consent	Records consent but does not enforce it downstream. We tie the record to enforcement and a hash chain.
ISO/IEC 29184:2020 [5]	Guidance on notices, consent and withdrawal	Guidance only. We implement versioned notices and incremental re-consent.
RFC 7519 / RFC 8725 (JWT) [6][7]	Signed tokens and security best practice	A token stays valid until expiry, so we add the cv freshness check.
RFC 7009 / RFC 7662 [8][9]	Token revocation and introspection	Introspection needs a network call per check. We use a local cv check against the cache.
RFC 6962; Schneier–Kelsey [10][11]	Append-only, verifiable logs	Merkle trees are heavy for a prototype. We use a per-user hash chain plus a verifier endpoint.

Existing work specifies or records consent. None of the sources we reviewed ties it to request-time enforcement with verifiable evidence. ConsentGuard does.

Proposed Solution

Frontend React / Next.js

Consent card per purpose, one-click grant/withdraw, re-consent banner, audit export.

Backend Node.js + Express

State transitions, notice versions, ES256 assertion-token issuer, audit export + chain verifier.

Enforcement PartnerHub

enforceConsent middleware: live check on every request, 401/403 with a machine-readable reason.

Database MongoDB + Mongoose

consent_state, consent_events (append-only), notices, access_logs.

P1_ANALYTICS app usage events

P2_PERSONALISATION activity summary, preferences

P3_RESEARCH_SHARING pseudonymised heart-rate, sleep

From click to enforcement (three purposes, each with an independent lifecycle)

1 Frontend captures intent

The dashboard renders one card per purpose against the current notice version, with nothing pre-ticked. Grant/withdraw sends {purposeId, noticeVersion}, so what the user saw is what gets stored.

2 Backend processes the transition

It validates the notice version (409 NOTICE_OUTDATED if outdated) and atomically increments cv. It appends a hash-chained event , commits it, , updates the cache and only then returns 200.

3 Consumer obtains an assertion

PartnerHub requests a token from /assertions. The backend checks that consent is granted and the notice is current. If not, no token is issued. Otherwise it issues a 5-minute signed token (ES256) containing the user, purpose, notice version, cv and a hash of the allowed data scope.

4 Mock API intercepts the request

The PartnerHub API verifies the signature locally, checks user and purpose, and compares the token's cv to the live cv. It returns 200 with only the allowed data, 401 for a bad token, or 403 with a reason code.

Consent is a state machine. Enforcement checks that state at request time, not only at token-issue time.

Proposed Solution: 20 lifecycle tests (requirement: ≥ 12)

Grant & sharing

T01 Grant P1 with notice v1
T02 Grant P1, P2, P3 independently
T03 Stale notice → 409
T04 Token + P1 request → 200
T05 No consent → 403

Withdrawal

T06 Withdraw P2, cv increments
T07 Same token after withdraw → 403
T08 Withdraw P2, P1/P3 unaffected
T14 Re-grant: old token stale
T15 Double withdraw idempotent

Denial

T09 Wrong purpose → 403
T10 Tampered / expired token → 401
T16 Alice's token on Bob → 403
T20 Cache cleared: same decisions

Notice & audit

T11 Material change: only P3 re-prompts
T12 Re-consent on v2
T13 Typo change: no re-prompt
T17 Export, chain valid
T18 Tamper detected
T19 Denied logs flushed

Tooling: Jest + Supertest + mongodb-memory-server, with a fresh synthetic user per test. Demo path: T01 → T04 → T06 → T07 → T11/T12.

Proposed Solution: expected results

Test	Scenario	Expected result
T01	Grant P1 against notice v1	200 · GRANTED, cv=1, event stores noticeVersion=1
T04	Valid token requests P1 data	200 · payload limited to P1 data categories
T06	Withdraw P2 after granting	200 · WITHDRAWN, cv incremented, event appended
T07	Reuse valid unexpired P2 token right after withdrawal	403 CONSENT_WITHDRAWN (proves instant enforcement)
T11	Publish v2 with a material change to P3	403 NOTICE_OUTDATED on P3, P1/P2 unaffected, re-prompt flagged
T12	Re-consent P3 against notice v2	200 GRANTED again, cv incremented, v1 history retained
T17	Export full audit trail and verify	chainValid: true across the complete lifecycle

cv trace: grant cv=1 → share (token cv=1) 200 → withdraw cv=2 → same token 403 CONSENT_WITHDRAWN → notice v2: only P3 re-prompts. This is the required demo: grant → data sharing → withdrawal → denied request → notice change.

Proposed Solution: enforceConsent middleware

```
async function enforceConsent(req, res, next) {
  // 1. local crypto check, no I/O
  claims = jwt.verify(token, pubKey, ES256)
    else 401 INVALID_TOKEN
  if claims.sub != userId        -> 403 SUBJECT_MISMATCH
  if claims.purpose != purpose   -> 403 PURPOSE_MISMATCH

  // 2. live state: cache, then Mongo
  state = cache.get(key) ?? db.findOne(key)

  // 3. decide + freshness
  d = decide(state, purposeMeta)    // status, notice
  if !d.allow                  -> 403 d.reason
  if claims.cv != state.cv     -> 403 TOKEN_STALE

  // 4. allow, log async
  auditQueue.push(ALLOWED); next()
}
```

Illustrative pseudocode

Withdraw

findOneAndUpdate {status: GRANTED} → WITHDRAWN with $inc cv. Append a hash-chained event, then write the cache, then return 200.

Assertion

decide() must allow, otherwise 403 and no token exists. The token is a 5-min ES256 JWT with sub, purpose, nv, cv, scope.

Audit chain

hash = SHA256(prevHash + event). The verifier detects tampering by reporting the first broken sequence index.

Optimization Proposed

< 5 ms auth check: cache + local JWT verify

0 s withdrawal effect: cv increases + cache write

Async access-log batching, consent events stay sync

Optimization	Standard (Before)	Optimized (After)	How
API authorization latency	~45 ms, DB read on every call	**< 5 ms**	Write-through in-memory cache (userId:purposeId), a miss falls back to Mongo. JWT verified locally, no network call.
Withdrawal propagation	Waits for token expiry or a nightly sync	**Effective at acknowledgement**	cv bump + cache write before the 200, older tokens fail the live cv check.
Audit log overhead	Sync DB write blocks the request	**Async batched access logs**	Background queue for access logs, consent events stay synchronous and hash-chained, so evidence is never dropped.
Audit integrity	Mutable rows	**Per-user SHA-256 hash chain + verifier**	hash = SHA256(prevHash + event),first broken seq reported.
Notice version change	Full re-authentication	**Incremental re-prompt of changed purposes only**	material changes computed per purpose at notice publication.
Lifecycle testing	Sequential, manual	**20 automated Jest tests, parallel, isolated users**	Fresh synthetic user per test, Jest + Supertest + mongodb-memory-server.

Solution Architecture & Design

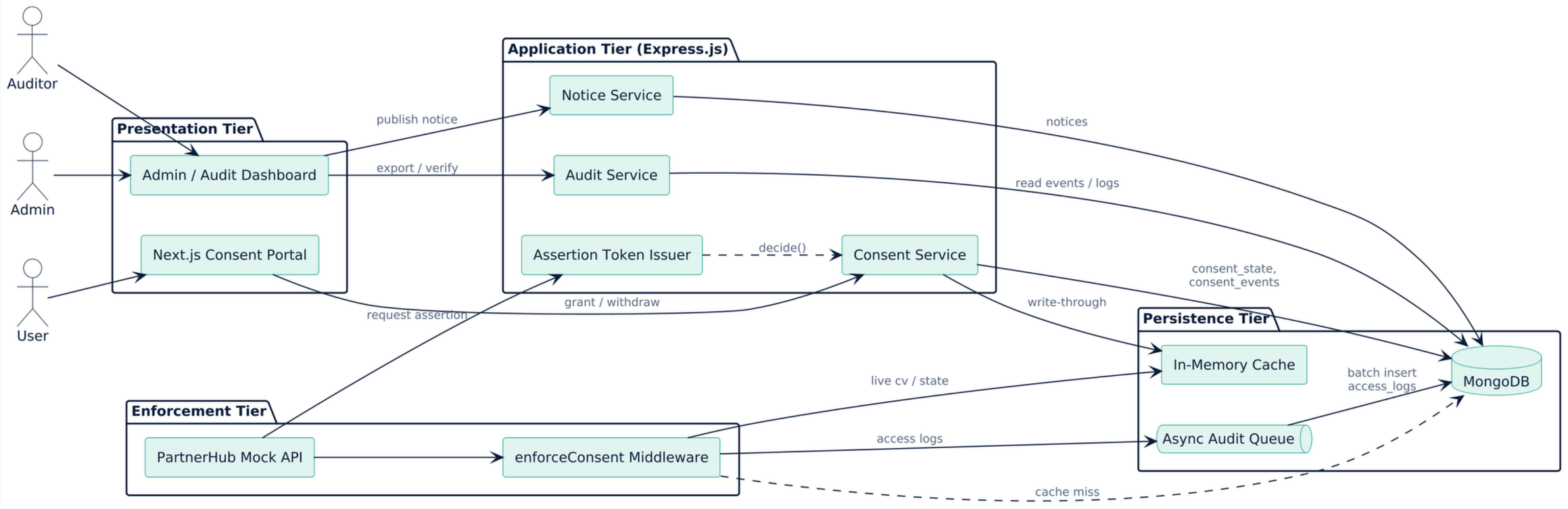


Figure: System Architecture UML Diagram

Four tiers : **Presentation**: Next.js/React portal and admin dashboard.
Application: Express REST API (auth, notices, lifecycle rules, tokens).
Enforcement: PartnerHub, the only path to data.
Persistence: MongoDB.

Four collections :
consent_state (fast read model) ·
consent_events (append-only ledger, source of truth) ·
notices (immutable, versioned) ·
access_logs (async allow/deny log).

Data Flow Diagram

Legend navy box = external entity · teal oval = process · D = data store

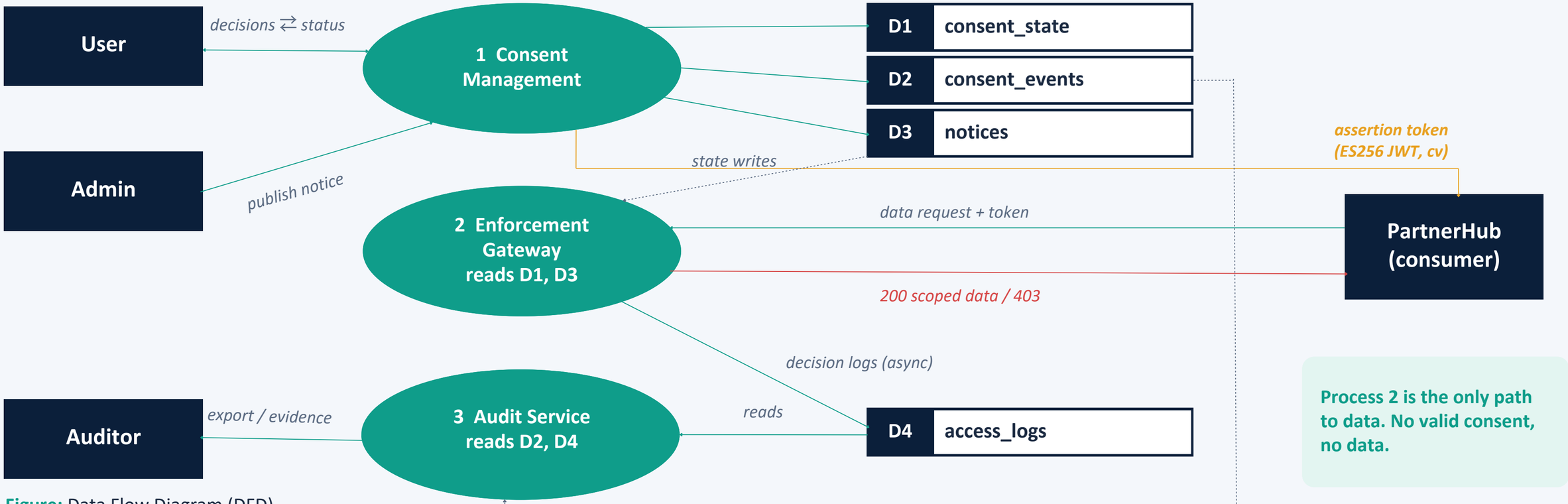
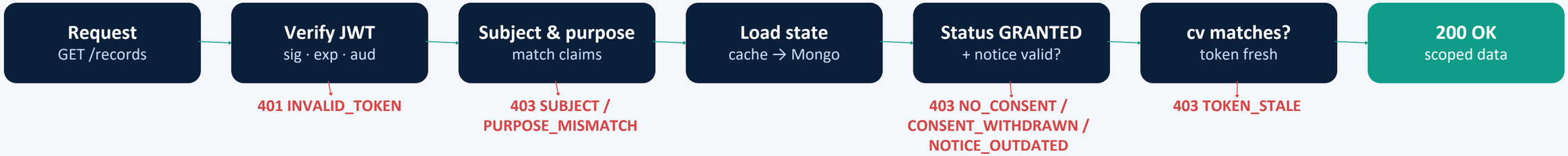


Figure: Data Flow Diagram (DFD)



Deployment Timeline

Phase	Dates	Work	Milestone
1. Architecture & Schema	*[dd Mon – dd Mon]*	Mongo models for multi-purpose consent and notice versions; API contracts for the mock consumer	**M1**: schema and contracts frozen
2. Backend & Middleware	*[dd Mon – dd Mon]*	Grant, withdraw and notice routes; PartnerHub mock consumer + enforceConsent	**M2**: enforcement returns 200/401/403 with reason codes
3. Frontend & Tests	*[dd Mon – dd Mon]*	Next.js consent portal, 20 lifecycle tests (≥ 12 required)	**M3:** all tests green; demo path rehearsed
4. Optimize & Submit	*[dd Mon – dd Mon]*	Cache tuning, audit-export check, GitHub repo, final PDF	**M4:** repo transferred to the AIORI account, PDF submitted

Methodology **API-contract-first, iterative; enforcement layer built test-first with Jest/Supertest; changes reviewed via GitHub pull requests.**

Bibliography

[1] Government of India, “The Digital Personal Data Protection Act, 2023 (No. 22 of 2023),” Aug. 2023.
[2] MeitY, Government of India, “Digital Personal Data Protection Rules, 2025,” notified Nov. 2025.
[3] European Parliament and Council, “Regulation (EU) 2016/679 (General Data Protection Regulation),” *Off. J. Eur. Union*, L 119, 2016.
[4] Kantara Initiative, “Consent Receipt Specification, v1.1,” 2018.
[5] ISO/IEC 29184:2020, “Information technology — Online privacy notices and consent,” 2020.
[6] M. Jones, J. Bradley, N. Sakimura, “JSON Web Token (JWT),” RFC 7519, IETF, May 2015.
[7] Y. Sheffer, D. Hardt, M. Jones, “JSON Web Token Best Current Practices,” RFC 8725, IETF, Feb. 2020.
[8] T. Lodderstedt, S. Dronia, M. Scurtescu, “OAuth 2.0 Token Revocation,” RFC 7009, IETF, Aug. 2013.
[9] J. Richer (Ed.), “OAuth 2.0 Token Introspection,” RFC 7662, IETF, Oct. 2015.
[10] B. Laurie, A. Langley, E. Kasper, “Certificate Transparency,” RFC 6962, IETF, Jun. 2013.
[11] B. Schneier, J. Kelsey, “Secure audit logs to support computer forensics,” *ACM Trans. Inf. Syst. Secur.*, vol. 2, no. 2, pp. 159–176, 1999.

Thank You

Access is stopped when the consent is Withdrawn

ConsentGuard demonstrates how purpose-specific consent, version tracking, withdrawal handling, and tamper-evident audit records can help make data-sharing workflows more transparent and accountable.

Thank You for your time and consideration.

Team No Means No
Team code :A3-PS002-TC140

Aliza Parvez Yousuf | Md Samir Khan

Code: github.com/khanSamir-jmi/A3-PS002-TC140